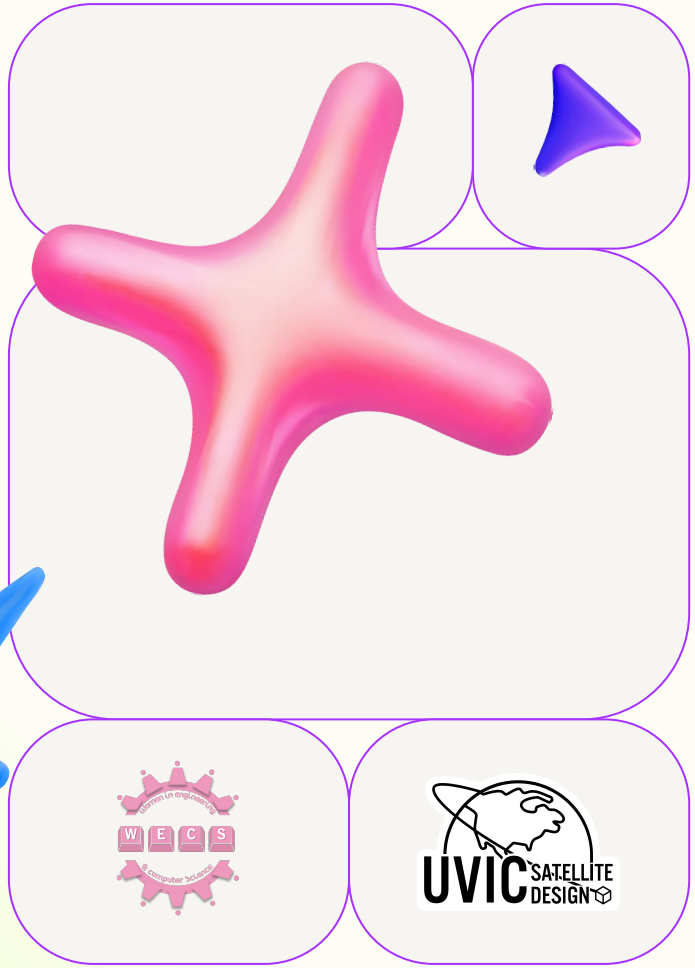
WECS & UVSD

Intro to the Command Line and Git

July 25, 2026



Agenda

1. Intro to the Command Line
2. Intro to Version Control & Git
3. Advanced Git topics: Conflicts & Rebasing
4. More topics: Make your own Git repo, WSL, Customize Your Shell

The Command Line

A Command Line Interface (CLI)

Lets you interact with software using only text commands, with no visual interface.

Many software programs have a CLI, including your operating system!

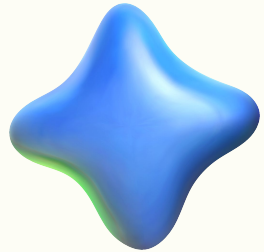
What is the command line on your computer?

A way to interact with your computer using text commands instead of the user interface. Also called the terminal.

What can you do with it?

Everything you can do with your desktop environment (and maybe more).

A note on operating systems



Windows

Windows Command Prompt (CMD) and PowerShell are built in, Windows 11 comes with the Terminal app which runs both.

Linux

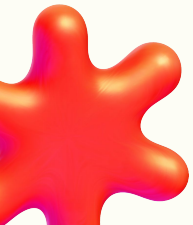
Based on Unix! Your built in terminal may depend on your Linux distribution.

MacOS

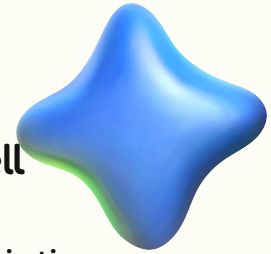
Based on Unix! The built in Terminal is exactly what it's called.

Other...

There are many many OSs, each with possibly different terminals! You may need to do some research.



A note on shells



Sh

The original shell! No one uses it anymore

Bash

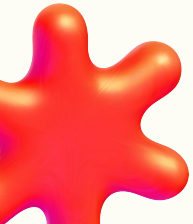
“Bourne Again Shell” - the original shell but made better

Zsh

A more modern version of bash

PowerShell

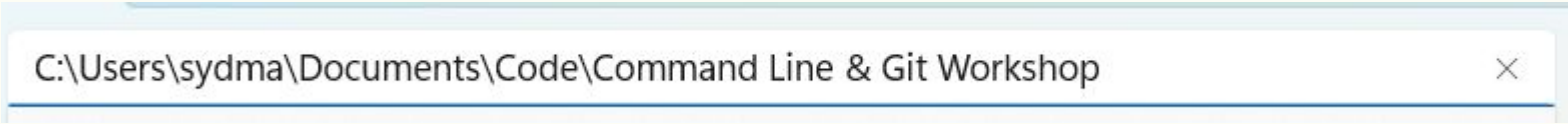
Its own scripting language and is a little different from other shells under the hood. Don’t worry about it.



★The commands you use depends on the terminal language (shell)★

File paths

The place where a file is stored on your computer is called a ‘file path’ – i.e. the path you need to take to get to your file.



Special “files”

“ . ” refers to the current directory
“ . . ” refers to the parent directory

Root

Root is the top of your file system.

On Unix, it is called root (not to be confused with the root user, who has ultimate permissions) and is at `/` in the file system.

On Windows, this would be the top of your C drive at `C:\`.

Home

Your home directory is the top of your user-specific file system.

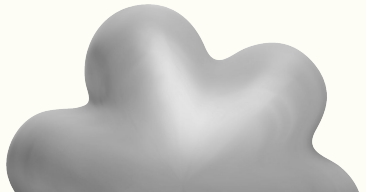
On Windows, this is at `C:\Users\<name>`. On Unix, it is at `/home/<name>`. Also called `~`.

Some useful commands

>`cd <dir>` for changing directories (folders)
>`ls` for listing directory contents (`dir` is the native Windows command)
>`cp <current path> <new path>` for copying
>`mv <current path> <new path>` for moving or renaming

`--help` is your friend

(often, but not always, can be abbreviated to `-h`)



Useful when you don’t know how to use a command

Exercise: navigating in the terminal

1. Open a terminal!
2. Find the same location in your file explorer app
3. What's here?
    a. Unix: Use >ls
    b. Windows: Use >dir
4. Move to a new directory (folder). Use >cd <folder>

Exercise: navigating in the terminal ctd.

5. Choose a place on your computer to store the files from today
6. From the terminal, create a new folder. Use >`mkdir <folder name>`
7. See your new folder in your file explorer?
8. Make a new file from your file explorer
9. See your new file in the terminal?

Hidden files

Some files are “hidden” and don’t show up in your file explorer or in `ls` output

These files are prefixed with “.”

Example: `.bashrc`, `.config`, `.gitignore`

We can view these files using the `-a` flag

`.` and `..` are hidden files which are present in every directory!



Vim & Vi & Nano

Text editing from the terminal!

These are programs which allow you to open a file and edit it, much like Word, Notepad, VS Code, etc, except that all interaction (like saving the file, copying and pasting) is done through text commands.

For this workshop we will use Vim. To install vim on Windows: https://www.vim.org/download.php

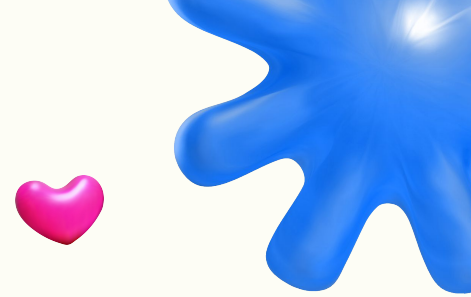
Vi, Vim, and Nano come pre-installed on most Linux and MacOS distributions, as well as Git Bash on Windows.

★ To learn more, run >`vimtutor` in your terminal!

Exercise: Write to a file

1. Make a new file from the terminal. Use >`vim <file name>`
    a. On Unix, you can use `touch <file name>` to create a file if it doesn't exist already
2. Add something to the file
    a. Vim/Vi: Type `i` to enter insert mode
3. Save your changes and exit the file
    a. Vim/Vi: `ESC` (to exit insert mode), `:w` to write changes, `:q` to quit the file (use `:wq` to save and exit at the same time!)
    b. Nano: `Ctrl+O` to save changes, `Ctrl+X` to exit

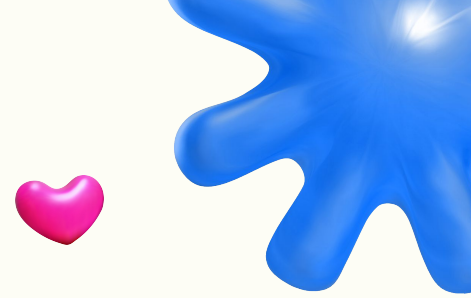
Running programs



Unix	To execute some program/file, it must have execute permissions
Windows	To execute some program/file, it must have an executable file type (.exe, .cmd, etc.) or be a binary

How to run?	Simply use the name as a command
	`>program.exe`
	`>executable-file`
	`>./another-program`

Flags & Arguments



--version, -V	A flag! Essentially a toggle for some behaviour
--version=2.0.0	A flag with a value!
Command line arguments	Pass some data to the program

Exercise: Make a program

Put the following into a file

1. `echo The contents of my repo: > contents.txt`
2. `ls git-workshop >> contents.txt`
3. `cat contents.txt`
4. `rm contents.txt`

★ PowerShell: make sure the file has a `.ps1` extension

★ Unix: you may need to run >`chmod +x` to make the file executable (view permissions with >`ls -l`)

The PATH Variable

The PATH variable contains the paths to files or programs you need regularly. When the shell is looking for something, it checks the locations listed in the PATH variable.

If the file or program is not in your PATH, your shell may not be able to find it.

Exercise: Update your PATH for this session

Unix:

1. Run `export`

   ```
   PATH=$PATH:/file/path
   ```

Windows:

1. Run `set`

   ```
   PATH=%PATH%;C:\your\path\
   ```

These changes will only last during your current session, and will disappear once you close the shell session

Exercise: Update your PATH permanently

Unix:

1. Open ~/.bashrc or ~/.zshrc
2. Add `export PATH=$PATH:/file/path`
3. Run >`source ~/.bashrc` or restart terminal

Windows:

1. Open your control panel
2. Click “Environment variables”, then “New”
3. Add your variable and apply changes
4. Or if you are **very sure**, run `setx PATH=%PATH%;C:\your\path\`

File permissions (Unix)

Files in Unix-like systems have a set of permissions which specify how different users can interact with the file, specifically if they can read, write, or execute it.
Permissions look like this when displayed with `ls -l`:
-rwxrwxrwx
Which specify the file type, owner permissions (u), user group permissions (g), and other permissions (o)

`chmod`: command change the permissions on a file by using the letter corresponding to the user group, + or - to set/unset, and r, w, or x for the action you want to alter
For example: `chmod g-wx` to remove write and execute permission from the user group.

For more details, a pretty good explanation can be found here.

Super user (Unix)

The super user (or root user) has ultimate permissions on a Unix-like system, similar to an administrator.

sudo: execute the following command with super user permissions. Sudo is password protected and will prompt for a password

★ Fun fact: don't run sudo on any UVic computers or servers – it won't work and will send a ticket to IT

Some more practice

Continue exploring the terminal with the MIT Terminus game: https://web.mit.edu/mprat/Public/web/Terminus/Web/main.html

Questions?

Version Control & Git

What is version control?

A way to track how information changes over time and sometimes revert to an older version.

For example: Google Docs history, numbering versions of a file, or (a cool one) the Internet Archive

What is Git?

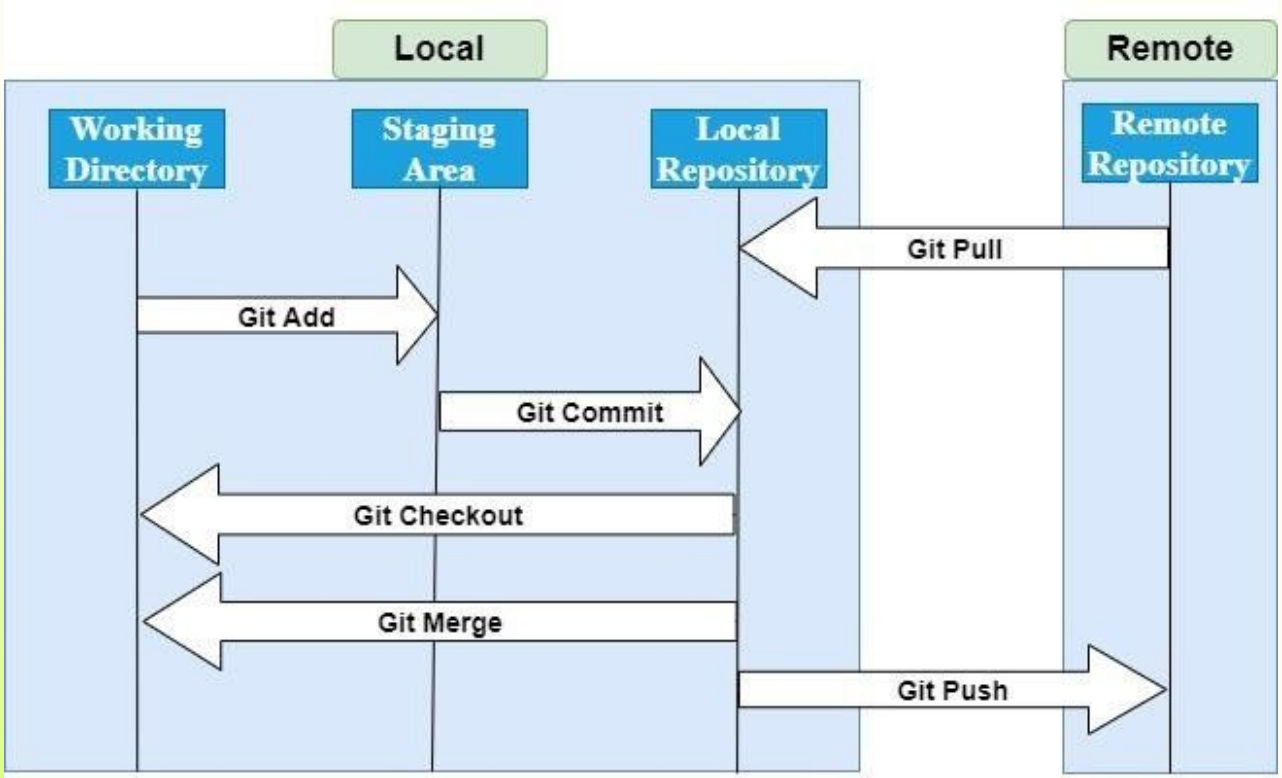
Git is a distributed version control software (VCS)

- Tracks concurrent changes to files from different sources
- Ensures single source of truth

★ A great resource for learning Git basics: https://git-scm.com/book/en/v2

The basic Git workflow

(With a remote repository)



- A **local repo** is your personal copy of the git repo
- The **remote repo** is the central version of the project
- The **working directory** is where you actively edit your project files.
- The **staging area** holds selected changes, preparing them to be saved to the project history.

Common commands

`git clone`: Make a copy of a repository

`git init`: Create a new repository

Exercise: Clone a repository

1. Go to github.com/uvic-wecs/git-workshop
2. Go to the repository called `git-workshop`
3. Click the “Code” button and copy the URL
4. Now go to your desired directory in your terminal
5. Run >`git clone <URL>` with the URL you copied
6. Cd into your new repo and look around!



Branching & merging

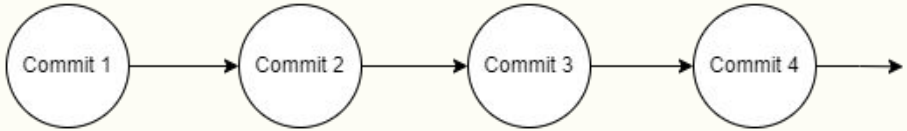
Think of the repo as a linear progression of commits. If more than one person is modifying a specific file, there may be conflicts between their changes. They may edit the same line, and git will not know how to combine, or merge, the changes together.

A branch is a particular version of the repo that “branches” off into a new progression of commits. This allows developers to make changes concurrently without conflicting with others’ work.

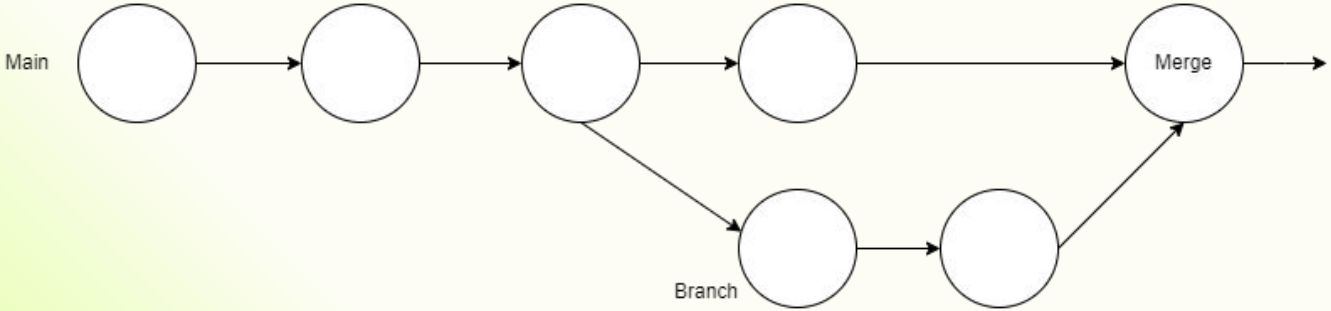
When finished making this batch of changes, the different versions of the repo can be combined, or merged, back into the main progression, and all conflicts can be resolved at this time.

Branching & merging

Linear:



Branching:



Branching & merging

Common practice in industry is to not commit directly to the “main” branch.



Common commands

`git branch`: View all branches in your repository
`git branch <name>`: Create a new branch

`git checkout`: Change branches
`git checkout -b <name>`: Change to a new branch

`git switch`: Change branches cleverly (Git will do some extra work, like set that branch to track an upstream one)

Exercise: Make a branch

1. Back in your repository, create a new branch using `git checkout -b <branch name>`
2. Make a change to a file in today's folder
3. Attempt to switch back to `main` using `git checkout main`, observe what happens
4. Undo your changes and try to switch branches again

Common commands

`git status`: Check the status of your repo and current changes

`git diff <file>`: See the difference between your working copy of a file and the last committed version, or between different commits

Exercise: Make some changes

1. Make sure you are on your own branch!
2. In today's folder in the repo, make a new file. Include your name in the new file name
3. Add some text to the file and save it
4. Run `git status` and observe the result
5. Run `git diff <file name>` with your new file and see your changes

Common commands

`git add <path>`: Stage changes in your repo

`git commit`: Commit all staged changes, you will be prompted for a message to describe the commit
`git commit -m <message>`: Include your commit message in the command

`git push`: Push changes from your local repo to the remote one
`git push -u <remote> <branch>`: Push a local branch to the remote, you must specify which remote and the branch name (which must match the local branch)

`git log`: View the commit history

Exercise: Commit the changes

1. Stage the changes you just made using `git add <file>`
2. Commit your changes to your branch using `git commit`. Add a commit message when prompted and save it - remember how to use terminal text editors from earlier!
3. Run `git push -u origin <branch name>` to push your branch and new commit to the remote repo
4. See your commit in the Git history with `git log`

Merging & pull requests

When merging a branch, it’s important to review the changes that are being integrated into the central code source.

On GitHub, we do this by making a **pull request**. This provides an interface where the code changes can be viewed on GitHub and feedback provided.

Exercise: Make a pull request

1. Back to the remote repo on GitHub!
2. Make a pull request from your branch to `main`
    a. A pop up may appear to automatically create this pull request
    b. If not, go to the “Pull requests” tab, click “New pull request”. Choose `base: main` and `compare: <your branch>`
3. Click “Create pull request”
4. Let’s review! Introduce yourself to someone, find their PR, and approve their changes.

Common commands

`git revert`: Undo a commit by making exactly opposite changes

`git restore`: Undo changes which aren't staged/committed, or use with `--staged` to unstage changes you've added

`git stash`: Save current untracked changes, but remove them from the working directory. The changes are pushed to a stack, and can be added back with `git stash apply`

`git pull`: Pull changes from the remote into your local branch, pull changes from a different branch by specifying the remote and branch name

Exercise: Sync the repo

1. Once all pull requests are merged, go back to your local repo
2. Switch to the main branch (`git checkout main`) and run `git status`, Git should let you know you are behind the remote
3. Run `git pull` to update your branches with everyone's changes

Questions?

Conflicts & Changing History in Git

Conflicts

Since Git doesn’t automatically or continuously keep your local repo in sync with other copies of the repo, sometimes we run into what we call **conflicts**.

This occurs when changes made in different branches, or by different contributors, overlap or contradict each other.

This typically happens during a merge or pull request when two branches have modified the same part of the codebase in incompatible ways.



Fixing conflicts

When you attempt to merge code and there is a conflict, Git will place the conflicting code in a block that looks like this:

```
<<<<<<< HEAD
Some changes
=======
Different changes
>>>>>>> other-branch
```

You can fix the conflict by choosing what changes to keep (this could be a mix of both changes), removing all `< = >` characters, then saving a committing the file.

Exercise: Resolve a conflict

1. Back in your local repo, pull any changes from main and **don't pull anymore**!
2. Make some changes to conflict.txt in today's folder and commit your changes to a branch
3. I will push a new change to conflict.txt on `main`
4. **Now** pull the changes into your branch with `git pull origin main`. There should be a conflict!
5. Fix the conflict in your editor of choice by choosing which changes should stay, and commit the merge

Editing the Git history

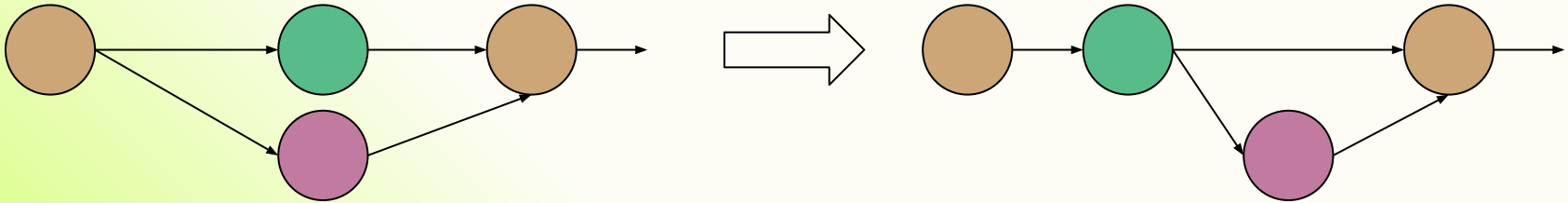
Even after making a commit, your changes are not set in stone. You can alter previous commits and their messages (for example, with `git commit --amend`), or change the order commits are applied, or even combine multiple commits.

Rebasing

Rebasing is when Git edits the commit history.

Rebasing on a specific commit essentially moves some commits after a later commit and applies the changes as if they were made to the later version of the repo (shown in the diagram). This can cause conflicts.

During a rebase, you can also make different edits like amending or squashing commits, changing commit messages, and more.



Squashing

Squashing is when multiple commits are merged into one commit with all of the changes. You can choose commits to be squashed during a rebase and can edit the commit message.

This can be useful if you want one commit per merge, one commit per feature added, or to squash fixup commits, as just some examples.

Common commands

`git rebase`: Edit the commit history. Include the `--interactive` flag to manually edit the history, and specify the remote and branch name or commit to rebase on a specific commit.

Exercise: Rebase

1. Make a new branch from `main`, make some changes, and commit
2. I will push a new commit to `main`
3. Run `git rebase --interactive origin/main` and follow instructions in the file that opens (no changes should be needed)
4. This will apply your commits after the new changes on main (there may be merge conflicts which you need to resolve)
5. Now the changes should appear sequentially in the git history

Questions?

Make Your Own Git Repo

Exercise: Make a new repo

1. Find a directory on your computer you want to turn into a Git repo. Could be class work, personal project, or empty!
2. Run `git init`. This will turn the current folder into a repo tracked by Git by adding a `.git` folder (this is where all Git information is stored)
3. Try your new favourite Git commands out

Exercise: Push your new repo to GitHub

1. Head to your GitHub profile and create a new repo with the “New” button under the Repositories tab
2. Make sure you don’t add any files when creating the repo. You want the “add README.md” and “add .gitignore” boxes to be **unchecked**
3. Copy the URL of your new **empty** repo
4. Back in your local repo, run `git remote add origin <url>`. This will add your GitHub repo as a remote (or upstream) called `origin`.
5. Now you can push your local files to the remote repo

Questions?

Windows Subsystem for Linux

WSL

Windows Subsystem for Linux (WSL) lets you run a mini Linux operating system on your windows computer. You can run different Linux distributions and can even have multiple on your computer at once.

You can install WSL by running `wsl --install` on PowerShell, see more details at https://learn.microsoft.com/en-us/windows/wsl/install.

WSL can be super useful for running certain programs that work better on Linux, especially for programming.

Questions?

Customize Your Shell

Customize your shell appearance

Use tools like

- Starship (works on a lot of popular shells)
- Oh My Posh (works on Windows, MacOS, Linux)
- Oh My Zsh (for Zsh): framework to manage your shell, change the theme, add tools, etc.
- Ghostty (terminal emulator which you can customize)

Install tools

`tldr`: neat tool that will summarize the lengthy unix man pages

`bashmarks`: make shortcuts in your shell

Questions?

Thank you!

Find the command cheat sheet and these slides at
github.com/uvic-wecs/git-workshop

Watch out for more workshops at
@uvicwecs
@uvicsatellitedesign